Model 1 Outline

Abstract

This is an outline for my first model. The goal was the exercise the different components I learned about in Prince's Understanding Deep Learning. It was a successful experiement, and good practice. I have two goals the next two models will build on. First, I want to experiment with synthetic data, the last few chapters of the book were about generative models, so I want to try building a generative model for different bits of these two datasets. Second, I want to refine this model a bit, get some stronger results, then attach a conformal predictor to it, as further practice. Since finishing Prince's book, I have begun reading "a gentle introduction to conformal prediction and distribution-free uncertainty quantification".

1 Objective

The primary object is to use the pre-training plus fine-tuning paradigm, ontop of some kind of transformer architecture. The most obvious choice was to do win-prediction, hence the goal is win prediction.

Ultimately, this sub-project is about selecting and justifying an architecture (for the given task, which may not be well posed), and empirically learning about the effects of choices like optimizers, loss functions, regularization techniques, and subtle architecture adjustments. All is in the service of spurring on my own interest in the field.

2 Data

The data for this project comes from a public Deadlock data API. It comes in two forms:

1. End-of-Match - This is essentially a screenshot of the final scoreboard. This will include variables determined by the outcome of the game, like kills, death, items, and souls, but will also have element invariant of outcome, like character selection, lane assignment, and rank (elo).

2. Event Stream - This is literally the entire game. Every entities position at every tick of the game, every characters soul count, kills, deaths, assists, spatial position, ability cooldowns, etc. This is the ultra-high resolution spatial-temporal data that initial inspired me to consider this dataset.

These two datasets provide a nice form factor for the pre-training/fine-tuning paradigm. The first, End-of-Match data, is extremely small, and easy to get, providing the usual medium for pre-training. The second, Event Stream data, provide, again, an excellent structure for this paradigm; Extremely high resolution, expensive, well labeled data. However, the event stream data also has the bonus feature that the ultra-high detail of it allows for multimodal use of the data.

3 Architecture

The architecture I came up with is as follows:

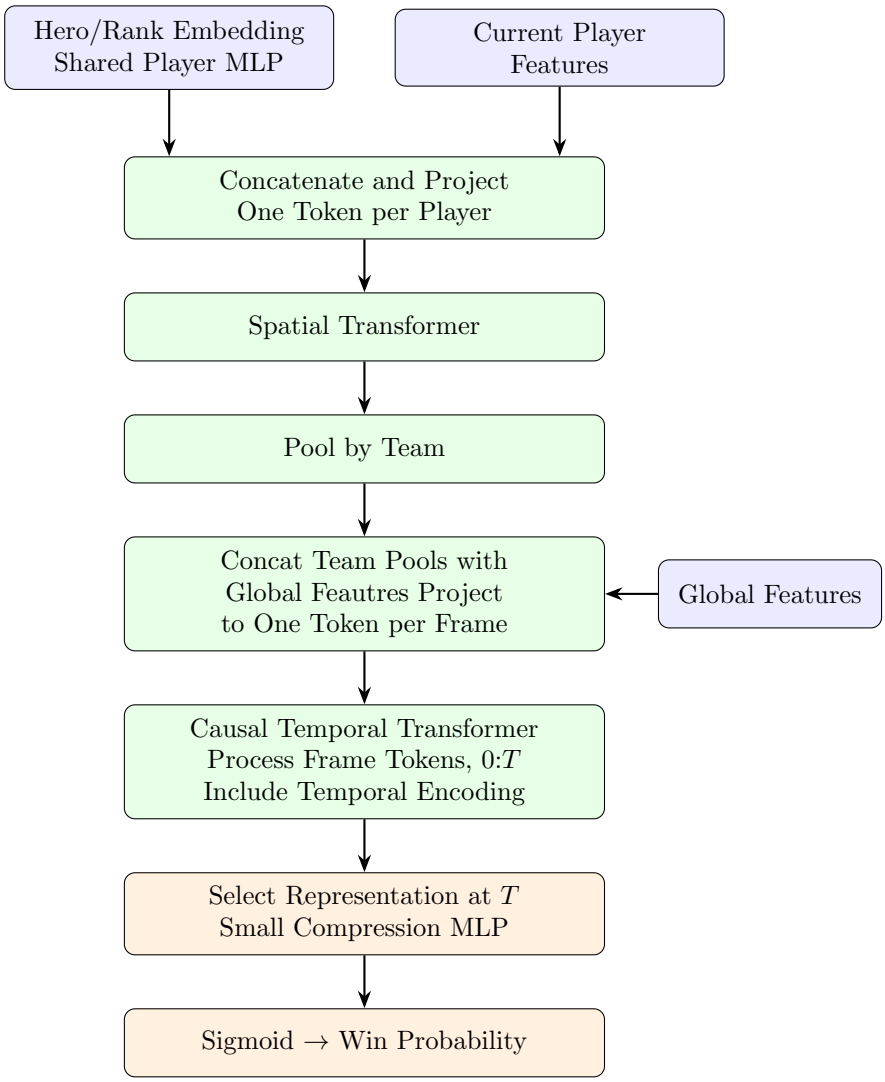


Figure 1: Win-prediction architecture. Shared player encoding and spatial attention are applied to each frame; temporal attention combines the observed history to predict the final match outcome.

Let $\mathcal{I}$ denote the set of heroes, e.g., $\mathcal{I} = \{\text{Abrams}, \text{Apollo}, \text{Warden}, \ldots\}$. For a given match, let $p \in \{1, \ldots, 12\}$ index players and $t \in \{0, \ldots, T\}$ index observed frames. Player p selects hero $i_p \in \mathcal{I}$ and has scaled pre-match rank $r_p \in \mathbb{R}_+$. Let $\mathbf{x}_{p,t} \in \mathbb{R}^F$ contain current player features, including team identity, position, kills and death, and let $\mathbf{g}_t \in \mathbb{R}^G$ contain global features like tower damage, urn count, elasped time. Let $[\mathbf{a}; \mathbf{b}]$ denote vector concatenation.

Component	**Purpose**	**Symbol & Structure**
Hero embedding	Learned hero lookup vector.	$e_i \in \mathbb{R}^{16}$ for each $i \in \mathcal{I}$
Static player MLP	Condition hero representation on rank.	$\mathbf{s}_p = f_{\text{static}}([\mathbf{e}_{i_p}; r_p]) \in \mathbb{R}^{32}$ where f_{static} is an MLP with ReLU layers MLP: $17 \to 32 \to 32$
Player projection	Combine static and current features.	$\mathbf{z}_{p,t} = \mathbf{W}_{\text{player}}[\mathbf{s}_p; \mathbf{x}_{p,t}] + \mathbf{b}_{\text{player}} \in \mathbb{R}^{64}$
Spatial transformer	Contextualize players within a frame.	$\mathbf{U}_t = \text{Spatial}(\mathbf{Z}_t) \in \mathbb{R}^{12\times 64}$; 1 block, 4 heads, feed-forward with width 128.
Team pooling	Summarize each team's players.	$\mathbf{a}_{k,t} = \frac{1}{6}\sum_{p\in\mathcal{P}_k} \mathbf{u}_{p,t} \in \mathbb{R}^{64}$, where $\mathcal{P}_k$ is team k's player set.
Frame projection	Combine team summaries and global state.	$\mathbf{v}_t = \mathbf{W}_{\text{frame}}[\mathbf{a}_{1,t}; \mathbf{a}_{2,t}; \mathbf{g}_t] + \mathbf{b}_{\text{frame}} \in \mathbb{R}^{64}$.
Temporal encoding	Represent frame order.	$\tilde{\mathbf{v}}_t = \mathbf{v}_t + \mathbf{q}_t$, where $\mathbf{q}_t \in \mathbb{R}^{64}$ is a positional encoding.
Temporal transformer	Represent observed match history.	$\mathbf{h}_T = \text{Temporal}(\tilde{\mathbf{v}}_0, \ldots, \tilde{\mathbf{v}}_T)_T \in \mathbb{R}^{64}$; 2 blocks, 4 heads, feed-forward with width 128.
Win head	Predict team 1's win probability.	$\hat{p}_T = \sigma(f_{\text{win}}(\mathbf{h}_T))$; where f_{win} is an MLP with ReLU layers $64 \to 32 \to 1$

Table 1: Proposed architecture. $\mathbf{Z}_t$ stacks player tokens as rows; $\mathbf{u}_{p,t}$ is player p's spatial output.

Now with the architecture figured out, how to train it becomes the next major question.

4 Training

In order to meet the goals of this sub-project, and take advantage of all the available data, this is the following training procedure I have come up with for this model. Note that numerical values for parameters are initial quantities and will likely be adjusted during training.

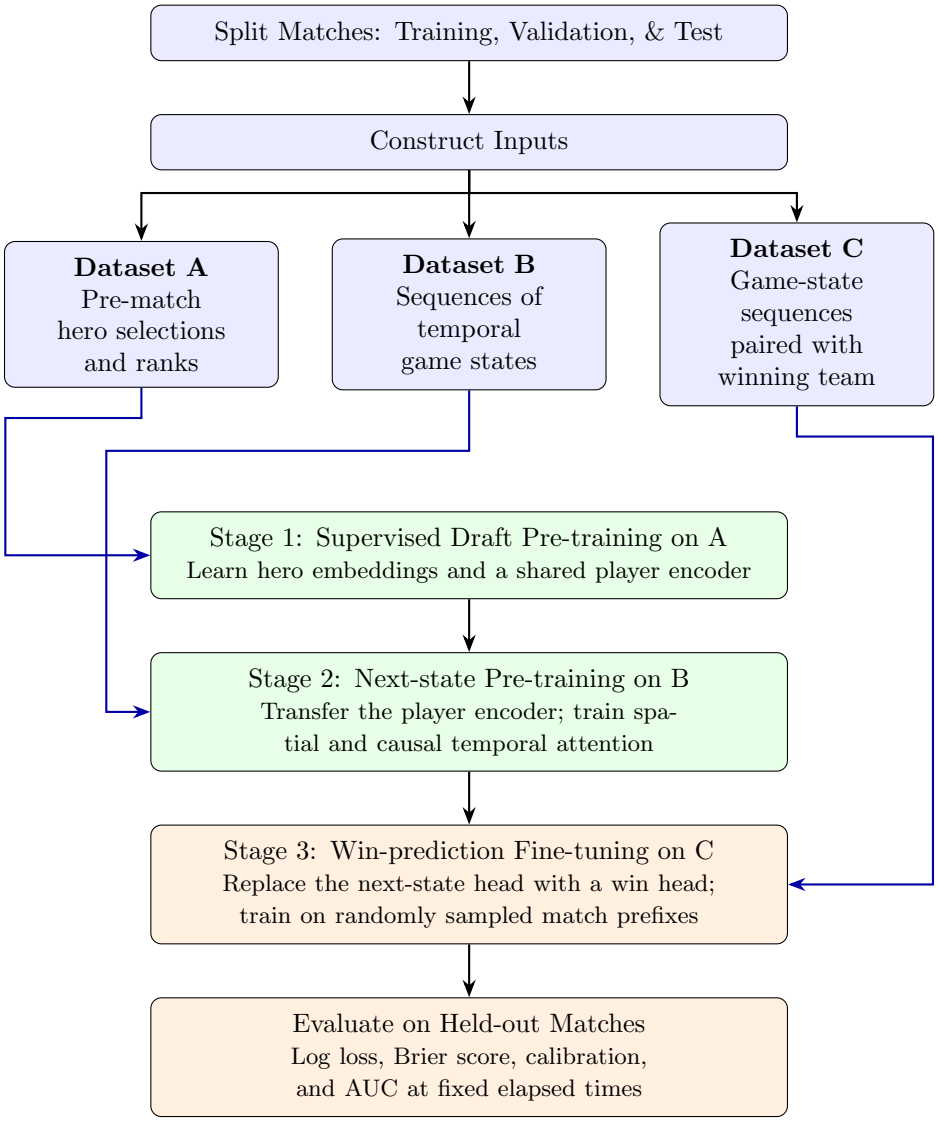


Figure 2: Proposed training procedure. Blue arrows supply each stage with its corresponding dataset; black arrows between training stages indicate parameter and data transfer. Datasets B and C may use the same trajectories, with different training targets.

Using this high-level overview, I now consider each step in more detail.

4.1 Pre-Train 1 - Static Embeddings

In this first stage, I want to learn hero embeddings and a shared MLP that combines hero identity with pre-match rank. I then use these static player representations in the later training stages.

4.1.1 Architecture

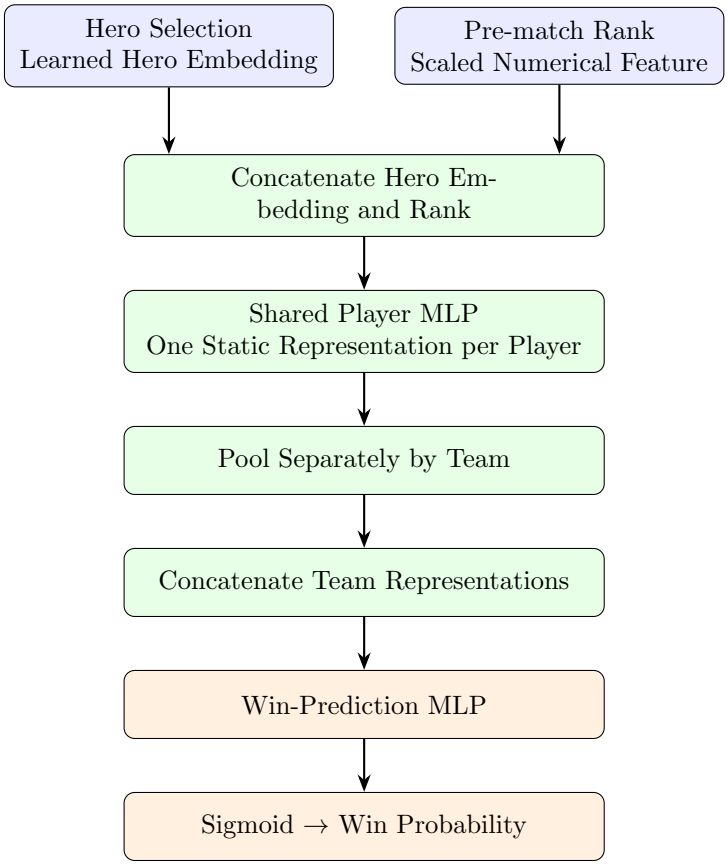


Figure 3: Draft-based pre-training architecture. Hero embeddings and the shared player MLP are trained using match outcomes and transferred to Stage 2. Everything after the pooling is discarded.

4.1.2 Loss function

Since this is a win-prediction task, I use binary cross-entropy between the predicted probability that team 1 wins and the observed outcome. For the sake of numerical stability, loss is computed directly from the output logit using `BCEWithLogitsLoss`.

4.1.3 Optimizer and learning rate

I use AdamW with an initial learning rate of 10^{-3} and initially set weight decay to zero. I may adjust the learning rate or introduce weight decay later based on validation loss.

4.1.4 Batch size and training duration

I use mini-batches of 64 matches and will train for at most 100 epochs, shuffling the training matches at each epoch. I retain the checkpoint with the lowest validation binary cross-entropy.

4.1.5 Regularization and stopping criteria

I halt if the validation loss does not improve for 10 consecutive epochs. Initially, I do not use dropouts but this may change if training loss continues to decrease while validation loss worsens.

Similarly, I initially do not use weight decay, but in the event of overfitting, I will.

4.2 Pre-Train 2 - Temporal Dynamics

For the second pre-training stage, I want the model to learn about game dynamics by predicting changes from the observed match history. I initialize the hero embeddings and shared player MLP from Stage 1, then train these alongside the new components.

4.2.1 Architecture

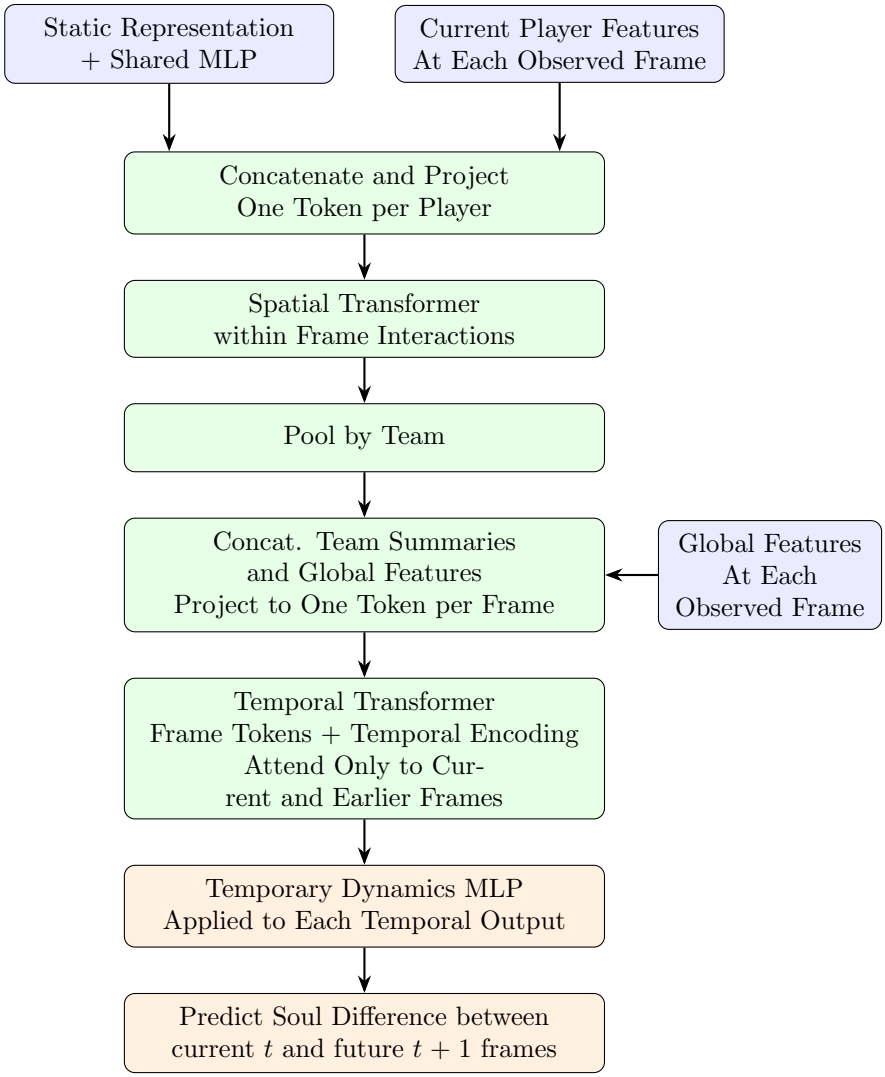


Figure 4: Dynamics pre-training architecture. The output at frame t predicts changes over the next recorded interval using only frames $0{:}t$. All components above the temporary dynamics head transfer to the final win-prediction model.

4.2.2 Prediction targets

I want to predict selected next-frame changes in continuous variables, such as position, souls, and health, together with next-frame binary states, such as whether a player is alive or an objective

remains standing. I exclude fixed attributes, like hero identity, and quantities I already know, like the next timestamp. For now, I start with a simple soul-difference prediction, but I may expand this later.

4.2.3 Loss function

I use mean squared error for continuous changes, which I standardize using training-set means and standard deviations. If I add binary targets, I will use binary cross-entropy computed from logits. I average each loss over its valid targets and exclude padded frames and missing targets. If I use both target groups, I will initially give them equal weight.

4.2.4 Optimizer and learning rate

I use AdamW with an initial learning rate of 10^{-4} for the transferred hero embeddings and static player MLP, and $\times 10^{-3}$ for newly initialized components. Initially I set weight decay to zero, but may introduce it if validation performance indicates overfitting.

4.2.5 Batch size and training duration

I start with mini-batches of 8 match sequences and will train for at most 100 epochs. I average prediction losses within each match before averaging across matches, so longer matches do not automatically receive more weight.

4.2.6 Regularization and stopping criteria

I use dropout of 0.1 within the transformer blocks and halt if validation loss does not improve for 10 consecutive epochs. I retain the checkpoint with the lowest validation loss and clip the total gradient norm to 1 to limit unusually large updates.

4.2.7 Causal masking and sequence handling

I use causal attention so the representation at frame t only uses frames $0{:}t$, then train it against targets from frame $t + 1$. I mask padding in both attention and loss computation, and omit prediction at the final frame since there is no subsequent observation.

4.2.8 Validation and parameter transfer

I compare prediction errors against a simple persistence baseline, which predicts no change from the current frame, and report results separately by target group. I then transfer the player encoder, spatial transformer, frame projection, and temporal transformer to the final stage, discarding the temporary prediction heads.

4.3 Fine-Tune - Win Prediction

Finally, I use the representations learned in Stage 2 to predict the winning team from a partially observed match. I replace the dynamics head with a randomly initialized win-prediction MLP, then fine-tune the entire model.

4.3.1 Architecture

I use the architecture in Figure 1, taking the temporal representation at the sampled cutoff T. I pass this through an MLP with dimensions $64 \to 32 \to 1$, with ReLU between layers and sigmoid to convert the final logit into a win probability.

4.3.2 Prediction targets and prefix sampling

I sample a training match uniformly, then sample a cutoff T uniformly among its eligible recorded frames and give the model only frames $0{:}T$. The target is the final match outcome, encoded as 1 if team 1 wins and 0 otherwise. I exclude terminal frames and any features that explicitly reveal the outcome.

4.3.3 Loss function

As in Stage 1, I use binary cross-entropy between the predicted win probability and the observed match outcome, averaged over sampled matches. I compute this directly from logits using `BCEWithLogitsLoss`, without applying sigmoid beforehand.

4.3.4 Optimizer and learning rate

I use AdamW with an initial learning rate of 10^{-5} for the transferred backbone and 10^{-4} for the new win-prediction head. Initially, I set weight decay to zero. As before, these learning rates are starting values that I may adjust based on validation loss.

4.3.5 Batch size and training duration

I use mini-batches of 8 match prefixes and will train for at most 100 epochs, sampling a new cutoff for each training match each epoch. I pad prefixes within each batch as needed and take the temporal output at each prefix's actual cutoff, rather than at a padded position.

4.3.6 Regularization and stopping criteria

I retain transformer dropout of 0.1 and gradient clipping at a total norm of 1. I halt if validation binary cross-entropy does not improve for 10 consecutive epochs, then restore the checkpoint with the lowest validation loss.

4.3.7 Causal masking and sequence handling

I retain causal attention and mask padded frames, so each representation only uses the available match history. I also reuse the feature scaling learned from training data; normalizing each match using its complete trajectory would give the model information about the future.

4.3.8 Validation

I keep the validation prefixes fixed so I can compare checkpoints without the added variation from new cutoffs. I report binary cross-entropy, Brier score, calibration, and AUC at fixed elapsed times, using only matches still in progress at each time.

5 Initial Results

This section is a both a summary of my completed first run, and some brainstorming for next step/improvements.

5.1 Embedding Pre-Train (Stage 1)

5.1.1 Summary Statistics

I trained the static encoder using only pre-match hero selections and ranks from the end-of-match database. Table 2 summarizes the saved run; the constant-probability reference is the BCE for predicting $p = 0.5$ on every match.

Statistic	**Value**
Training / validation / reserved test matches	10,528 / 1,316 / 1,316
Selected epoch / completed epochs	8 / 18
Validation BCE at selected checkpoint	0.673767
Constant $p = 0.5$ BCE reference	0.693147
Training BCE: selected epoch / final epoch	0.670678 / 0.667505
Validation BCE at final epoch	0.677387
Missing player ranks across all splits	91.19%
Matches with every rank missing	11,917 / 13,160
Training rank median / minimum / maximum	53 / 11 / 116

Table 2: Stage 1 results and rank availability for the saved October 23 run.

5.1.2 Issues

Rank information was very sparse, ¿90% of observations did not have a rank. Median imputation can only do so much. Also, validation loss worsened after the selected checkpoint, and training loss kept falling, so it was overfitting.

5.1.3 Improvement

I need a more complex embedding structure, or a different type of training objective. Their is obvious benefit to embedding clearly, hero selection is a strong predictor, but I need to find a way to get that information more reliably, and to use all the information available.

I must consider how I want to deal with the missing rank information. Either I can simply use the reduced number of runs, since I already seem to have more than I need, or I can remove rank.

I will certainly keep early stopping. But I am considering weight decay, dropouts, and learning rate decay.

5.2 Dynamics Pre-Train (Stage 2)

5.2.1 Summary Statistics

The objective I used was to predict the next one-second change in team soul lead, Δ(team 1 net worth$-$ team 0 net worth). I averaged squared error within each match and then across matches, so long matches did not receive more weight. Persistence predicts no change in the lead.

Validation metric	Model	Persistence
Standardized MSE	0.838067	0.953371
RMSE in souls	403.57	430.44
Relative MSE reduction	12.09%	—
Relative RMSE reduction	6.24%	—
Run statistics		
Training / validation / reserved test matches		970 / 116 / 134
Valid training / validation targets		2,031,046 / 241,431
Selected epoch / completed epochs		23 / 33
Training MSE: selected / final epoch		0.839863 / 0.825801
Validation MSE at final epoch		0.841365
Successful-run wall time		100.6 minutes

Table 3: Stage 2 results from the completed October 29 run. Wall time excludes the stopped initial attempt.

5.2.2 Issues

I had to make some adjustment to how the batches were loaded, as it would not fit in memory. Also, there was also clear overfitting, as the validation loss was increasing while training loss was decreasing.

5.2.3 Improvements

I think I'll try to reduce the learning rate for the transferred encoder. Because of overfitting, I also plan on trying dropouts, weight decay, and learning rate decay. I may also try adjusting the window of prediction from 1 second to 5 seconds.

5.3 Win-Prediction Fine-Tune

5.3.1 Summary Statistics

I conducted two runs of this fine-tune. So, I compared both runs on the same validation prefixes. Both started from the same Stage 2 checkpoint, used the same seed, and retained the architecture, preprocessing, match splits, and uniform per-match prefix sampling. I used the training win frequency as the constant baseline.

Validation metric	**Prior**	**Run 1**	**Run 2**
BCE (lower is better)	0.692897	0.638630	0.632603
Brier score (lower is better)	0.249875	0.224818	0.222922
AUC (higher is better)	0.500000	0.680952	0.686012
ECE, 10 bins (lower is better)	0.013118	0.083140	0.088072
Setting / run statistic		**Run 1**	**Run 2**
Backbone learning rate		10^{-5}	3×10^{-6}
Transformer dropout		0.1	0.2
Learning-rate scheduler		None	Plateau
Rate reductions after epochs		—	28, 31, 34
Selected / completed epochs		9 / 19	25 / 35
Wall time (minutes)		50.7	86.9
Training BCE at selected epoch		0.618717	0.601868
Final training BCE		0.561030	0.601634
Final validation BCE		0.658338	0.635913
Training / validation / reserved test		970 / 116 / 134 matches	
Run 2 relative BCE / Brier reduction		0.94% / 0.84%	
Paired 95% interval, BCE (Run 2 − Run 1)		$[-0.03008, +0.02209]$	

Table 4: Fine-tuning comparison. The descriptive interval uses 10,000 paired match bootstrap resamples (seed 2026). GPU workload was not controlled between runs, so wall time is not a controlled throughput comparison.

5.3.2 Issues

Both runs suffered from clear overfitting.

5.3.3 Improvements

The same as the both pretraining stages, I want to try dropouts, weight decay, and learning rate decay.